String matching: Naïve Approach

Session No.: 10
Course Name: Design and analysis of algorithm
Course Code: R1UC407B
Instructor Name: Mili Dhar
Duration: 50 Min.
Date of Conduction of Class:

Review of the key concepts

1. Review of previous session

Q: How can you quickly find a small word (substring) within a large paragraph?

Learning Outcome

Apply the algorithm to solve string matching problems

Analyse the time complexity of the algorithm.

Session Outline

1 Introduction to Brute force approach

2 Step by Step demonstration of Naïve approach

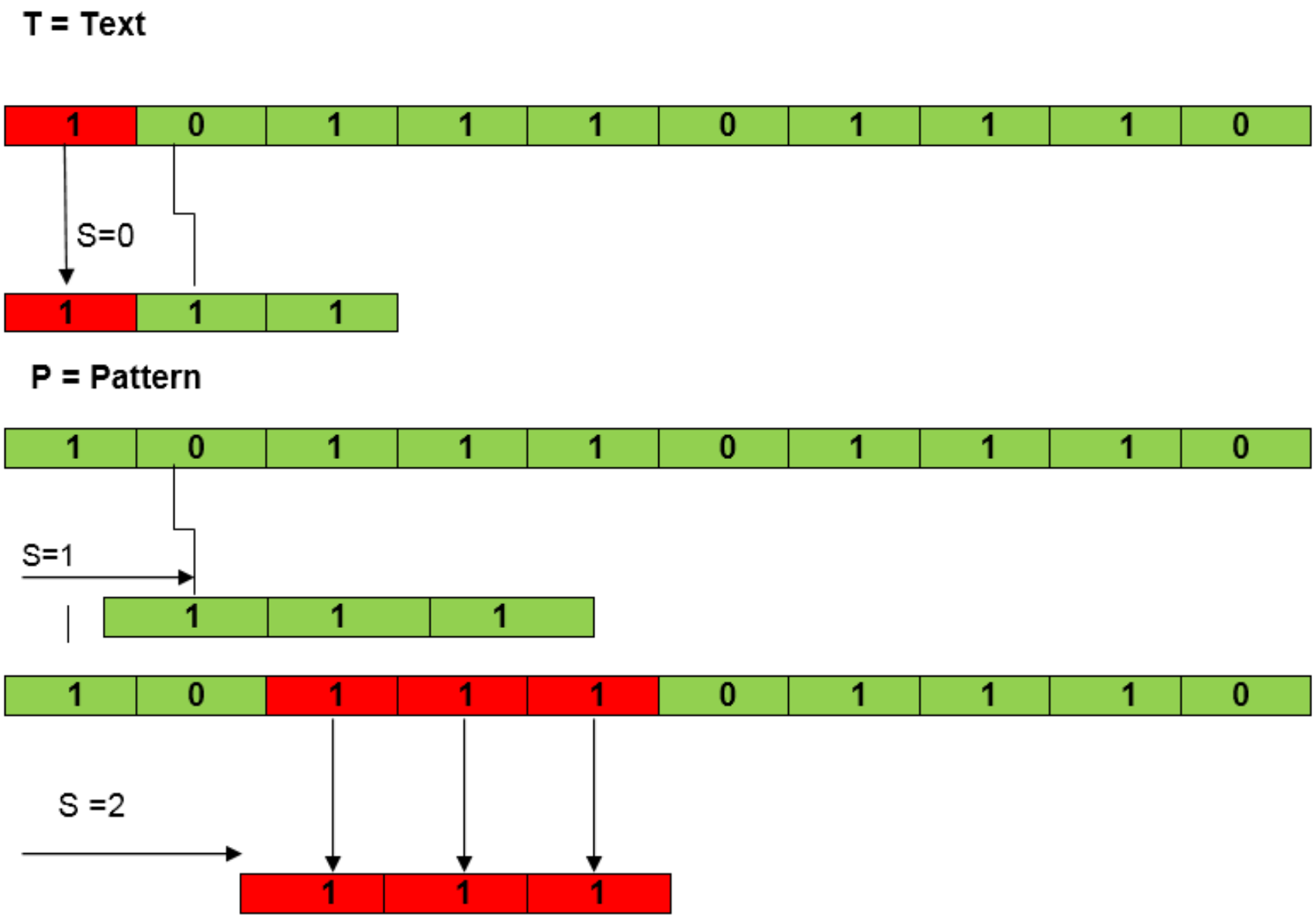
3 Apply Naïve approach to find a pattern in a given text

4 Analyse its complexity

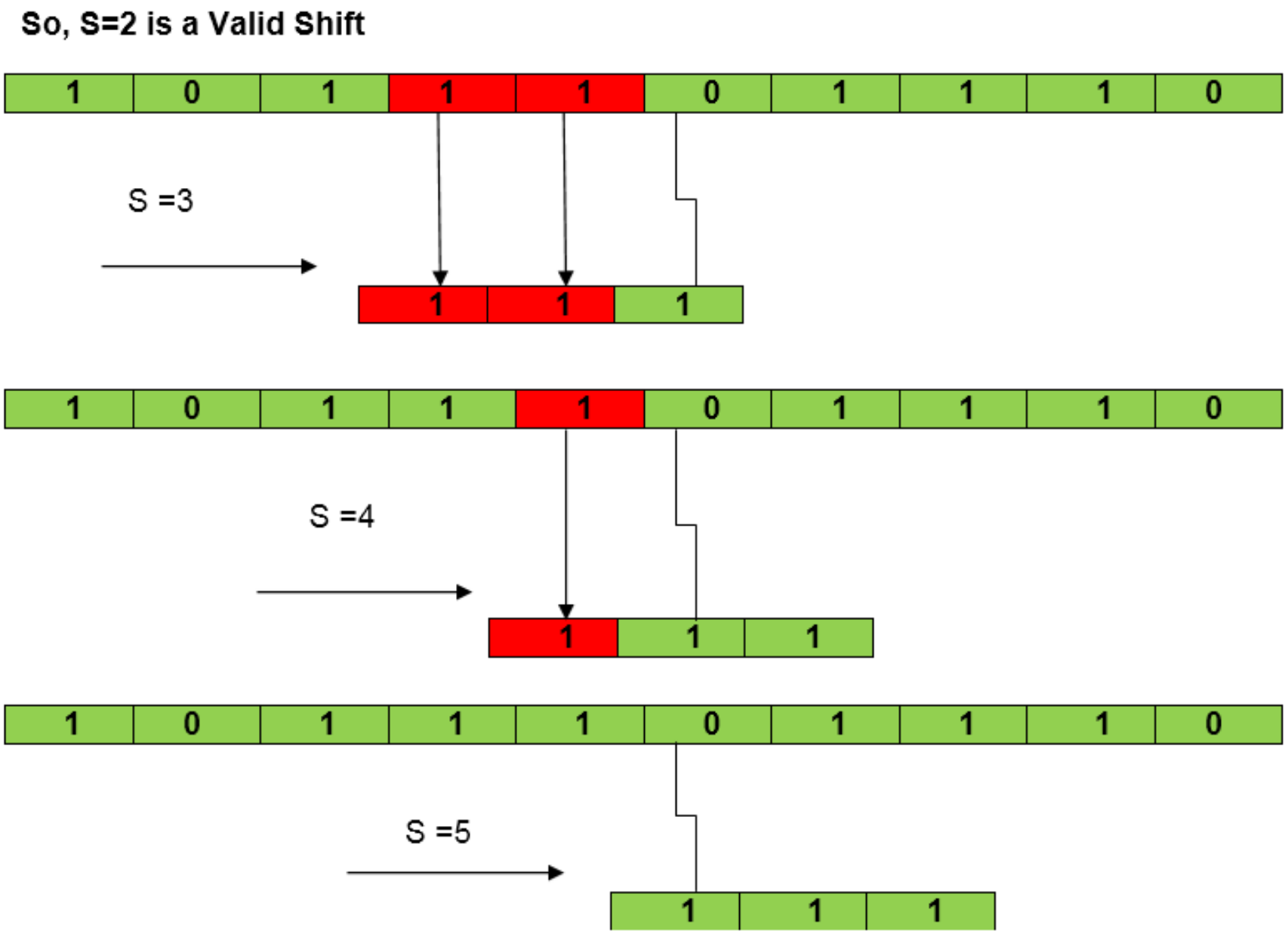
Naïve/Brute-Force Approach

1. Start at the beginning of the text and slide the pattern window over it.
2. At each position of the text, compare the characters in the pattern with the characters in the text.
3. If a mismatch is found, move the pattern window one position to the right in the text.
4. Repeat steps 2 and 3 until the pattern window reaches the end of the text.
5. If a match is found (all characters in the pattern match the corresponding characters in the text), record the starting position of the match.
6. Move the pattern window one position to the right in the text and repeat steps 2-5.
7. Continue this process until the pattern window reaches the end of the text.

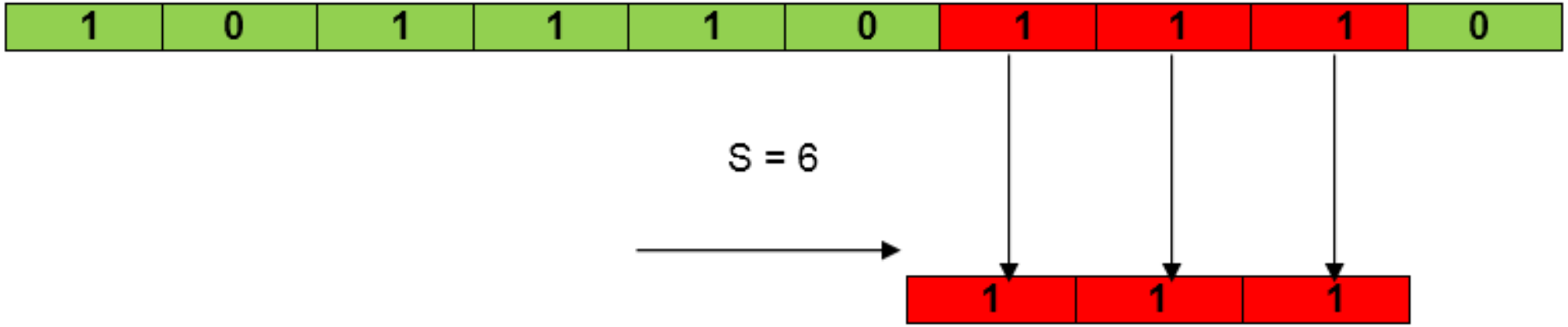
Demonstration



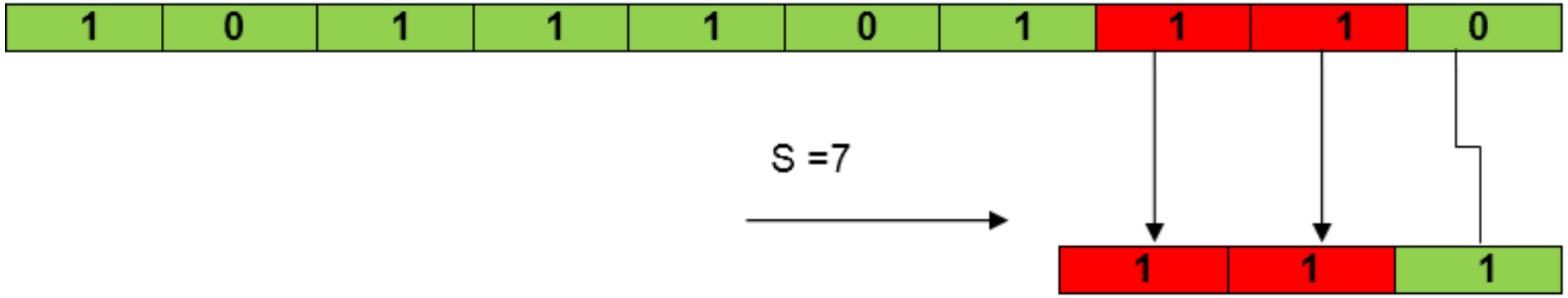
Demonstration



Demonstration



So, S=6 is a Valid Shift



Pseudocode of brute force method of string matching: Naïve method

1. NAIVE-STRING-MATCHER (T, P)
2. 1. n ← length [T]
3. 2. m ← length [P]
4. 3. for s ← 0 to n -m
5. 4. do if P [1.....m] = T [s + 1....s + m]
6. 5. then print "Pattern occurs with shift" s

Activity 2: Apply naïve method to the given scenarios

Input:

Main String: ABAAABCDBBABCDDEBCABC

Pattern: ABC

Analyse the Complexity of the followings:

Example:1
txt = "BBACCAADDEE";
pat = "HBB";

Note: The best case occurs when the first character of the pattern is not present in text at all. The number of comparisons in best case is O(n-m).

Example 2:
txt[] = "DDDDDDDDDDDD";
pat[] = "DDDDD";

Example 3:
txt[] = "VVVVVVVVVVVVVK";
pat[] = "VVVK";

The worst case of Naive Pattern Searching occurs in the following scenarios.

1. When all characters of the text and pattern are same.
2. When only the last character is different.

The number of comparisons in the worst case is O(m*(n-m+1)).

Post class assessment:

```
txt[] = "VVVVVVVVVVVVVK";
pat[] = "VVVK";
```

In the next class we will going through Rabin Karp approach

Thank You